

CHAPTER 10 COMPANION GUIDE

Full-Stack Web Development

From Absolute Beginner to Production-Ready Engineer

Complete coverage of:

HTTP & REST APIs • Node.js • Express.js
MongoDB • Mongoose • Authentication
bcrypt • JSON Web Tokens (JWT)
WebSockets • Real-Time Features
RabbitMQ • Message Queues
Microservices Architecture

No prior experience required — every concept explained from first principles

CHAPTER 1**How the Internet Works***The foundation: clients, servers, and the HTTP conversation***What Happens When You Type a URL?**

Every time you open a browser and type 'google.com', a remarkable sequence of events unfolds in milliseconds. Understanding this sequence is the foundation of everything else in this guide.

1. **DNS Lookup:** Your computer asks a DNS (Domain Name System) server: 'What is the IP address of google.com?' DNS is the internet's phone book. It converts human-readable names into numbers like 142.250.80.46.
2. **TCP Connection:** Your computer opens a reliable connection to that IP address on port 80 (HTTP) or 443 (HTTPS). This is a three-way handshake: SYN → SYN-ACK → ACK.
3. **HTTP Request:** Your browser sends a formatted text message — the HTTP request — asking for the page.
4. **HTTP Response:** Google's server sends back the page content, along with status codes and headers.
5. **Rendering:** Your browser interprets the HTML, CSS, and JavaScript to show you the page.

Anatomy of an HTTP Request

An HTTP request is just text with a very specific format:

HTTP Request Format

```
GET /search?q=javascript HTTP/1.1
Host: www.google.com
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64)
Accept: text/html,application/xhtml+xml
Accept-Language: en-US,en;q=0.9
Connection: keep-alive

(empty body for GET requests)
```

Line 1 — The Request Line: GET is the method, /search?q=javascript is the path+query, HTTP/1.1 is the version.

Lines 2–5 — Headers: Key-value pairs that give the server extra information: who is asking, what format they accept, etc.

Blank line: Separates headers from the body.

Body: Absent for GET requests. Present for POST/PUT when you send data.

HTTP Methods — The Verbs of the Web

HTTP methods tell the server what you want to DO, not just where to go:

Method	Purpose	Has Body?	Example
GET	Fetch/read data	No	GET /users → list all users
POST	Create new data	Yes	POST /users → create a user
PUT	Replace data entirely	Yes	PUT /users/5 → replace user 5
PATCH	Update part of data	Yes	PATCH /users/5 → update one field
DELETE	Remove data	No	DELETE /users/5 → delete user 5

HTTP Status Codes

Every response has a status code that tells you what happened. They are grouped by the first digit:

Range	Meaning	Common Examples
2xx	Success	200 OK, 201 Created, 204 No Content
3xx	Redirect	301 Moved Permanently, 302 Found
4xx	Client Error	400 Bad Request, 401 Unauthorized, 403 Forbidden, 404 Not Found
5xx	Server Error	500 Internal Server Error, 503 Service Unavailable

REST — Representational State Transfer

REST is a set of conventions for designing HTTP APIs. A RESTful API uses HTTP methods and URLs to represent resources and actions. Resources are nouns (users, games, products). Methods are verbs (GET, POST, DELETE). For example: GET /games → list games. POST /games → create a game. GET /games/42 → get game 42. DELETE /games/42 → delete game 42. Following REST makes your API predictable and easy to use.

CHAPTER 2

Node.js — JavaScript on the Server

How JavaScript escaped the browser and conquered the backend

What is Node.js?

For most of its life, JavaScript could only run inside a web browser. In 2009, Ryan Dahl took the V8 JavaScript engine (the one Chrome uses) and built Node.js — a way to run JavaScript anywhere: on your computer, on a server, in the cloud.

This was a big deal. Developers who knew JavaScript for the frontend could now write servers too. One language, everywhere.

The Event Loop — How Node.js Handles Many Users at Once

Traditional servers (like those written in Java or PHP) create a new thread for every request. A thread is like a worker. If 1000 users connect simultaneously, you need 1000 workers — expensive and slow to create.

Node.js uses a single thread with an event loop. Instead of waiting around while a database query runs, it starts the query, registers a callback ("call me when done"), and handles other requests in the meantime. When the database responds, the event loop picks up the callback and continues.

Blocking vs Non-Blocking

```
// Blocking (BAD in Node.js — hangs the whole server):
const data = readFileSync('data.txt'); // freezes everything!
console.log(data);

// Non-blocking (GOOD — Node.js way):
fs.readFile('data.txt', 'utf8', (err, data) => {
  // This runs LATER, when the file is ready
  console.log(data);
});
// Node.js continues here immediately without waiting
console.log('This prints BEFORE the file content!');
```

npm — The Package Manager

npm (Node Package Manager) lets you download and use code that others have written. There are over 2 million packages available — from database drivers to utility functions to full frameworks.

Common npm Commands

```
# Start a new project (creates package.json)
```

```
npm init -y

# Install a package (adds to node_modules/ and package.json)
npm install express

# Install as development dependency only (not needed in production)
npm install --save-dev nodemon

# Install all dependencies from package.json (after cloning a project)
npm install

# Run a script defined in package.json
npm run start
```

package.json — Your Project's Identity Card

```
package.json

{
  "name": "my-chess-server",
  "version": "1.0.0",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "build": "tsc"
  },
  "dependencies": {
    "express": "^4.18.2",
    "mongoose": "^7.0.0",
    "jsonwebtoken": "^9.0.0"
  },
  "devDependencies": {
    "nodemon": "^2.0.22",
    "typescript": "^5.0.0"
  }
}
```

Modules — Splitting Code Across Files

As your project grows, one file becomes unmanageable. Node.js has a module system to split code into files:

Node.js Modules (CommonJS)

```
// math.js — define and export functions
function add(a, b) { return a + b; }
function multiply(a, b) { return a * b; }

module.exports = { add, multiply }; // export what others can use

// _____

// app.js — import and use them
const { add, multiply } = require('./math'); // ./ = same folder

console.log(add(2, 3)); // 5
console.log(multiply(4, 5)); // 20
```

 TypeScript in Modern Projects

Many production projects (including the chess project this guide references) use TypeScript — a superset of JavaScript that adds static types. TypeScript catches errors at compile time instead of runtime. You write .ts files, run 'tsc' to compile them to .js, and Node.js runs the .js files. The syntax is identical to JavaScript with added type annotations like: `const port: number = 3000;` or `function greet(name: string): string { return 'Hello ' + name; }`

CHAPTER 3**Express.js — Building Web Servers**

From your first 'Hello World' to a full REST API

What is Express?

Node.js has built-in tools to create an HTTP server, but they are low-level and verbose. Express is a minimal framework that wraps Node.js's HTTP module and gives you a clean, simple API for handling requests.

Your First Express Server

server.js — Minimal Express Server

```
const express = require('express');
const app = express();

// Define a route: when someone GETs '/', respond with 'Hello World'
```

```
app.get('/', (req, res) => {
  res.send('Hello World!');
});

// Start listening for connections on port 3000
app.listen(3000, () => {
  console.log('Server running at http://localhost:3000');
});
```

Run this with 'node server.js', then open <http://localhost:3000> in your browser. You will see 'Hello World!'.

The Request Object (req)

The first argument to every route handler is the request object. It contains everything about the incoming request:

Exploring the Request Object

```
app.get('/demo', (req, res) => {
  console.log(req.method);           // 'GET'
  console.log(req.url);              // '/demo?name=alice'
  console.log(req.path);             // '/demo'
  console.log(req.query.name);       // 'alice' (from ?name=alice)
  console.log(req.headers);          // all HTTP headers as object
  console.log(req.ip);              // client's IP address
});

// Route parameters – the :id part becomes req.params.id
app.get('/user/:id', (req, res) => {
  console.log(req.params.id);        // '42' from URL /user/42
});

// POST route – req.body needs the JSON middleware (shown later)
app.post('/echo', (req, res) => {
  console.log(req.body);             // { name: 'alice', age: 25 }
});
```

The Response Object (res)

The second argument is the response object. You use it to send data back to the client:

Response Methods

```
app.get('/responses', (req, res) => {
  // Send plain text
  res.send('Hello!');

  // Send JSON (automatically sets Content-Type: application/json)
  res.json({ message: 'Success', user: { id: 1, name: 'Alice' } });

  // Send with a custom status code
  res.status(201).json({ message: 'Created!' });
  res.status(404).json({ error: 'User not found' });
  res.status(400).json({ error: 'Invalid input' });

  // Set a custom header
  res.setHeader('X-Custom-Header', 'my-value');

  // Redirect
  res.redirect('/new-location');
});
```

Middleware — The Heart of Express

Middleware are functions that run between a request arriving and a response being sent. They are the most powerful concept in Express. Every middleware function receives (req, res, next). It can read/modify req and res, then call next() to pass control to the next middleware. If it does NOT call next(), the request stops there.

Middleware Examples

```
// Custom middleware: logs every request
const logger = (req, res, next) => {
  const timestamp = new Date().toISOString();
  console.log(`${timestamp} ${req.method} ${req.url}`);
  next(); // MUST call next() or the request hangs forever!
};

// Apply globally (every request goes through this)
app.use(logger);

// — Built-in middleware —————

// Parse JSON request bodies (REQUIRED for req.body to work with JSON)
app.use(express.json());
```



```
// Parse URL-encoded form data (from HTML forms)
app.use(express.urlencoded({ extended: true }));

// Serve static files from a 'public' folder
app.use(express.static('public'));
```

The Middleware Pipeline

Think of middleware as a pipeline of water filters. Every request flows through ALL middleware in the order they are defined, until one of them sends a response. Order matters critically:

Middleware Pipeline Order

```
// Order matters! This is a full middleware pipeline example:

app.use(logger);           // 1st: log every request
app.use(express.json());   // 2nd: parse JSON bodies
app.use(authMiddleware);   // 3rd: check authentication

app.get('/protected', (req, res) => {
  // Only reaches here if auth passed
  res.json({ secret: 'you are authenticated!' });
});

// Error handling middleware - 4 parameters, always LAST
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ error: 'Something went wrong' });
});
```

Routing — Organising Your API

As your API grows, putting all routes in one file becomes messy. Express Router lets you split routes into separate files:

Express Router — Splitting Routes

```
// routes/users.js
const express = require('express');
const router = express.Router();

router.get('/', (req, res) => {
```

```
    res.json([{ id: 1, name: 'Alice' }]);
  });

router.get('/:id', (req, res) => {
  res.json({ id: req.params.id, name: 'Alice' });
});

router.post('/', (req, res) => {
  // Create new user from req.body
  res.status(201).json({ message: 'User created' });
});

module.exports = router;

// -----

// app.js - mount the router
const userRoutes = require('./routes/users');
app.use('/users', userRoutes);
// Now: GET /users → users.js '/'
// Now: GET /users/42 → users.js('/:id')
// Now: POST /users → users.js '/'
```

Error Handling

Unhandled errors crash your server. Good Express code always handles errors. Use try/catch in async routes and pass errors to next():

Async Error Handling Pattern

```
// WRONG - async error crashes the process
app.get('/bad', async (req, res) => {
  const data = await someFailingOperation(); // throws!
  res.json(data);
});

// CORRECT - catch and forward to error middleware
app.get('/good', async (req, res, next) => {
  try {
    const data = await someOperation();
    res.json(data);
  } catch (err) {
    next(err); // Express sends this to error middleware
  }
});
```

```
    }
  });

// Error middleware — always define LAST, always 4 params
app.use((err, req, res, next) => {
  console.error('Unhandled error:', err.message);
  res.status(err.status || 500).json({
    error: err.message || 'Internal server error'
  });
});
```

Building a Complete REST API

Let us put it all together — a fully working in-memory CRUD (Create, Read, Update, Delete) API for a list of users:

Complete CRUD REST API

```
const express = require('express');
const app = express();
app.use(express.json());

// In-memory 'database' (not persistent — resets on restart)
let users = [
  { id: 1, name: 'Alice', email: 'alice@example.com' },
  { id: 2, name: 'Bob', email: 'bob@example.com' },
];
let nextId = 3;

// GET /users — list all users
app.get('/users', (req, res) => {
  res.json(users);
});

// GET /users/:id — get one user
app.get('/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).json({ error: 'User not found' });
  res.json(user);
});

// POST /users — create a user
app.post('/users', (req, res) => {
```

```
const { name, email } = req.body;
if (!name || !email) return res.status(400).json({ error: 'name and email
required' });
const user = { id: nextId++, name, email };
users.push(user);
res.status(201).json(user);
});

// PATCH /users/:id — update a user
app.patch('/users/:id', (req, res) => {
  const user = users.find(u => u.id === parseInt(req.params.id));
  if (!user) return res.status(404).json({ error: 'User not found' });
  Object.assign(user, req.body); // merge updates
  res.json(user);
});

// DELETE /users/:id — remove a user
app.delete('/users/:id', (req, res) => {
  const id = parseInt(req.params.id);
  users = users.filter(u => u.id !== id);
  res.status(204).send(); // 204 No Content
});

app.listen(3000, () => console.log('API running on port 3000'));
```

Common Beginner Mistakes with Express

Forgetting `app.use(express.json())` — `req.body` will be undefined.

Not calling `next()` in middleware — the request hangs and eventually times out.

Sending multiple responses from one route (calling `res.json()` twice) — causes a 'Cannot set headers after they are sent' error.

Defining error middleware without 4 parameters — Express won't recognise it as error middleware.

Putting `app.use()` after routes — middleware defined after routes won't run for those routes.

CHAPTER 4

MongoDB — Storing Data That Lasts

Databases, documents, and why your data must survive a server restart

Why Do We Need a Database?

Earlier we built a CRUD API that stored users in a JavaScript array. This works perfectly — until you restart the server. Then all data is gone. A database is a program whose entire job is to store data permanently on disk, and to let you retrieve it quickly.

SQL vs NoSQL — Choosing the Right Tool

Feature	SQL (e.g. PostgreSQL)	NoSQL (e.g. MongoDB)
Data structure	Rigid tables with fixed columns	Flexible documents (like JSON objects)
Schema	Defined before inserting data	Schema optional, can vary per document
Relationships	JOINS between tables	Embedding or referencing
Best for	Financial data, structured data	User data, game state, flexible schemas
Query language	SQL	MongoDB Query Language (JSON-like)

MongoDB Core Concepts

MongoDB organises data in a hierarchy: Database → Collection → Document.

- **Database:** Like a folder. One MongoDB server can hold many databases (chess-db, blog-db, etc.)
- **Collection:** Like a table in SQL. A database can have many collections (users, games, moves).
- **Document:** A single record, stored as BSON (Binary JSON). Each document can have different fields.

Example MongoDB Document

```
// A MongoDB document looks exactly like a JavaScript object:
{
  "_id": ObjectId("64a1f2c3e4b0a1b2c3d4e5f6"),
  "username": "alice",
```

```
"email": "alice@example.com",
"passwordHash": "$2b$10$...",
"createdAt": ISODate("2024-01-15T10:30:00Z"),
"preferences": {
  "theme": "dark",
  "notifications": true
},
"gamesPlayed": [101, 205, 308]
}
```

💡 The `_id` Field

Every MongoDB document automatically gets an `_id` field — a unique identifier called an ObjectId. It is a 12-byte value that encodes a timestamp, machine ID, and random bytes. You never need to create IDs yourself unless you want custom ones. `ObjectId('64a1f2c3...').getTimestamp()` even tells you when the document was created!

Setting Up MongoDB

You have two options:

- **MongoDB Atlas (Recommended for beginners):** Free cloud-hosted MongoDB. Sign up at mongodb.com/atlas, create a cluster, and get a connection string like: `mongodb+srv://user:pass@cluster.mongodb.net/mydb`
- **Local Installation:** Download MongoDB Community Server. On Mac use Homebrew: `brew install mongodb-community`. Then start with: `brew services start mongodb-community`

CHAPTER 5

Mongoose — MongoDB with Superpowers

Schemas, models, and queries that feel like magic

What is Mongoose?

MongoDB is flexible — you can put any shape of data in any collection. That freedom is powerful but dangerous. Mongoose is an ODM (Object Document Mapper) library that adds structure, validation, and helper methods on top of MongoDB. It is the standard way to use MongoDB in Node.js.

Connecting to MongoDB

Connecting with Mongoose

```
const mongoose = require('mongoose');

// Connect — the URI tells Mongoose where MongoDB is and which database to use
mongoose.connect('mongodb+srv://user:pass@cluster.mongodb.net/chess-db')
  .then(() => console.log('Connected to MongoDB!'))
  .catch(err => {
    console.error('Connection failed:', err.message);
    process.exit(1); // quit if we can't connect
  });

// Better: store the URI in a .env file
// .env file: MONGO_URI=mongodb+srv://...
require('dotenv').config();
mongoose.connect(process.env.MONGO_URI);
```

Defining a Schema

A schema is a blueprint. It defines the shape, types, and validation rules for documents in a collection:

```
User Schema Definition

const mongoose = require('mongoose');
const { Schema } = mongoose;

const userSchema = new Schema({
  // Field: type, and optional constraints
  username: {
    type: String,
    required: [true, 'Username is required'],
    unique: true,           // no two users with same username
    trim: true,            // removes whitespace from both ends
    minlength: [3, 'Username must be at least 3 characters'],
    maxlength: [30, 'Username cannot exceed 30 characters'],
  },
  email: {
    type: String,
    required: true,
    unique: true,
    lowercase: true,       // auto-converts to lowercase
    match: [/^\S+@\S+\.\S+$/, 'Invalid email format'],
  },
  passwordHash: {
```

```
    type: String,
    required: true,
    select: false,          // NEVER returned in queries by default
  },
  rating: {
    type: Number,
    default: 1200,          // default chess ELO rating
    min: 0,
    max: 3000,
  },
  isActive: {
    type: Boolean,
    default: true,
  },
  createdAt: {
    type: Date,
    default: Date.now,      // auto-set to current time
  },
});

// Create the Model from the Schema
// 'User' → MongoDB collection 'users' (lowercase, pluralised automatically)
const User = mongoose.model('User', userSchema);

module.exports = User;
```

CRUD Operations with Mongoose

Mongoose CRUD Operations

```
// — CREATE —————

// Method 1: new + save()
const user = new User({ username: 'alice', email: 'alice@ex.com', passwordHash:
'...' });
await user.save(); // validates against schema, then inserts

// Method 2: create() — shorthand
const user2 = await User.create({ username: 'bob', email: 'bob@ex.com',
passwordHash: '...' });

// — READ —————
```



```
// Find all users
const allUsers = await User.find();

// Find with filter
const activeUsers = await User.find({ isActive: true });

// Find one
const alice = await User.findOne({ username: 'alice' });

// Find by ID
const user3 = await User.findById('64a1f2c3e4b0a1b2c3d4e5f6');

// Select specific fields (projection)
const usernames = await User.find({}, 'username email -_id');

// Include password hash (excluded by default due to select: false)
const userWithPw = await User.findOne({ username: 'alice'
}).select('+passwordHash');

// Sorting and pagination
const topPlayers = await User
  .find()
  .sort({ rating: -1 }) // -1 = descending
  .limit(10)           // take 10
  .skip(0);            // skip 0 (page 1)

// — UPDATE —————

// Update one field
await User.findByIdAndUpdate(userId, { rating: 1350 }, { new: true });
// { new: true } returns the UPDATED document, not the old one

// Use $set to update specific fields without overwriting others
await User.findByIdAndUpdate(userId, { $set: { rating: 1350, isActive: false }
});

// Increment a value
await User.findByIdAndUpdate(userId, { $inc: { rating: 50 } });
```

```
// — DELETE —————

await User.findByIdAndDelete(userId);
await User.deleteMany({ isActive: false }); // delete multiple
```

Query Operators — Filtering Like a Pro

```
MongoDB Query Operators

// Comparison operators
await User.find({ rating: { $gt: 1500 } }); // rating > 1500
await User.find({ rating: { $gte: 1500 } }); // rating >= 1500
await User.find({ rating: { $lt: 1200 } }); // rating < 1200
await User.find({ rating: { $ne: 1200 } }); // rating != 1200
await User.find({ rating: { $in: [1200, 1300] } }); // in this list

// Logical operators
await User.find({ $and: [{ isActive: true }, { rating: { $gt: 1500 } }] });
await User.find({ $or: [{ username: 'alice' }, { username: 'bob' }] });

// Text search (requires text index)
await User.find({ $text: { $search: 'alice' } });

// Regular expression
await User.find({ username: /^ali/i }); // starts with 'ali', case-
insensitive
```

Schema Virtuals and Methods

Mongoose lets you add computed properties (virtuals) and custom methods to your schema:

```
Virtuals and Methods

const gameSchema = new Schema({
  whitePieces: Number, // pieces remaining
  blackPieces: Number,
  startTime: Date,
  endTime: Date,
});

// Virtual: computed from other fields, not stored in DB
gameSchema.virtual('duration').get(function() {
  if (!this.endTime) return null;
```

```
    return (this.endTime - this.startTime) / 1000; // seconds
  });

// Instance method: available on document instances
gameSchema.methods.isOver = function() {
  return this.whitePieces === 0 || this.blackPieces === 0;
};

// Static method: available on the Model itself
gameSchema.statics.findActiveGames = function() {
  return this.find({ endTime: null });
};

const Game = mongoose.model('Game', gameSchema);

// Usage:
const game = await Game.findById(id);
console.log(game.duration); // virtual
console.log(game.isOver()); // instance method
const active = await Game.findActiveGames(); // static method
```

Middleware (Hooks) in Mongoose

Mongoose schemas support pre/post hooks — functions that run automatically before or after operations:

Mongoose Middleware Hooks

```
// Pre-save hook: runs BEFORE saving a document
userSchema.pre('save', async function(next) {
  // 'this' refers to the document being saved
  if (!this.isModified('password')) return next(); // skip if password
  unchanged
  // Hash the password before saving
  this.passwordHash = await bcrypt.hash(this.password, 12);
  this.password = undefined; // don't store plain password
  next();
});

// Post-save hook: runs AFTER saving
userSchema.post('save', function(doc) {
  console.log('User saved:', doc.username);
});
```

```
// Pre-find hook: runs before any find() query
userSchema.pre('find', function() {
  // Automatically filter out inactive users
  this.where({ isActive: true });
});
```

CHAPTER 6

Authentication — Who Are You?

Passwords, hashing, sessions, and proving identity securely

The Authentication Problem

HTTP is stateless — each request is independent. After you log in and the server sends a response, the connection closes. The next request has no memory of the login. So how does the server know who you are on subsequent requests? This is the authentication problem.

Never Store Passwords in Plain Text

This is the most important rule in authentication. If your database is ever breached and you stored passwords as plain text, every user's password is exposed. Instead, you store a hash — a one-way transformation that cannot be reversed.

⚠ Why MD5 and SHA-256 Are NOT Safe for Passwords

MD5 and SHA-256 are general-purpose hash functions designed to be FAST. An attacker with a GPU can try billions of passwords per second against them (brute force / rainbow tables).

bcrypt is specifically designed to be SLOW and to have a configurable 'cost factor'. Increasing the cost factor doubles the computation time. bcrypt also automatically adds a random 'salt' to prevent rainbow table attacks.

Always use bcrypt (or argon2) for passwords. Never MD5 or SHA-256.

Implementing bcrypt Password Hashing

```
bcrypt Password Hashing

const bcrypt = require('bcrypt');

// — HASHING (on registration) —————

const SALT_ROUNDS = 12; // cost factor — higher = slower = more secure
```

```
// 10 ≈ 100ms, 12 ≈ 400ms, 14 ≈ 1.5s per hash

const plainPassword = 'mysecretpassword123';
const hash = await bcrypt.hash(plainPassword, SALT_ROUNDS);
// hash looks like: '$2b$12$u8Ydm9y3....' (60 chars, includes salt)

// Save hash to database (never save plainPassword)
await User.create({ username: 'alice', passwordHash: hash });

// — VERIFYING (on login) —————

const userFromDb = await User.findOne({ username: 'alice'
}).select('+passwordHash');
const enteredPassword = req.body.password; // plain text from login form

const isMatch = await bcrypt.compare(enteredPassword, userFromDb.passwordHash);
// bcrypt extracts the salt from the hash and rehashes enteredPassword
// then compares — all internally. You just get true or false.

if (!isMatch) {
  return res.status(401).json({ error: 'Invalid credentials' });
}
// Proceed with login...
```

CHAPTER 7

JWT — JSON Web Tokens

Stateless authentication that scales to millions of users

What is a JWT?

A JSON Web Token is a self-contained, digitally signed token that proves identity. After a user logs in, the server creates a JWT containing the user's ID and signs it with a secret key. The client stores this token and sends it with every future request. The server verifies the signature — no database lookup needed.

JWT Structure

A JWT is three Base64-encoded parts separated by dots:

JWT Structure Explained

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQiOiI2NGExZjJjMyIsImhhdCI6MTY4OTQwMDAwMH0.xK8mN2pQ...
```

```
// Part 1 (Header) — algorithm and token type:
```

```
{ 'alg': 'HS256', 'typ': 'JWT' }
```

```
// Part 2 (Payload) — the claims (data you store in the token):
```

```
{ 'userId': '64a1f2c3e4b0', 'username': 'alice', 'iat': 1689400000, 'exp': 1689486400 }
```

```
// iat = issued at (Unix timestamp), exp = expiration
```

```
// Part 3 (Signature) — HMAC-SHA256(base64(header) + '.' + base64(payload), SECRET_KEY)
```

```
// This is what makes it tamper-proof — only the server knows SECRET_KEY
```

⚠️ JWTs Are Not Encrypted — They Are Only Signed

Anyone can decode the header and payload of a JWT (they are just Base64-encoded). Do NOT store sensitive data like passwords or credit card numbers in a JWT payload.

The signature proves the token was created by YOUR server and has not been tampered with. That is all it does.

Never put secrets in the payload. Only put identity information: userId, username, role.

Creating and Verifying JWTs**JWT: Create and Verify**

```
const jwt = require('jsonwebtoken');

const SECRET = process.env.JWT_SECRET; // must be long, random string

// — CREATING A TOKEN (on login/register) —————

function generateToken(userId, username) {
  return jwt.sign(
    // Payload — what you want to store in the token
    { userId, username },
    // Secret key — used to sign and later verify
    SECRET,
    // Options
    { expiresIn: '7d' } // token expires in 7 days
  );
}
```

```
}

// Usage:
const token = generateToken(user._id, user.username);
res.json({ token, user: { id: user._id, username: user.username } });

// — VERIFYING A TOKEN (on protected routes) —————

function verifyToken(token) {
  try {
    // Throws if token is invalid, expired, or tampered with
    const decoded = jwt.verify(token, SECRET);
    return decoded; // { userId: '...', username: '...', iat: ..., exp: ... }
  } catch (err) {
    return null; // invalid token
  }
}
```

Auth Middleware — Protecting Routes

```
authMiddleware.js

// authMiddleware.js

const jwt = require('jsonwebtoken');

// This is the Express middleware function
function authMiddleware(req, res, next) {
  // 1. Extract the token from the Authorization header
  //   Header format: 'Authorization: Bearer eyJhbGci...'
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = authHeader.split(' ')[1]; // 'Bearer TOKEN' → 'TOKEN'

  // 2. Verify the token
  const decoded = verifyToken(token);
```

```
if (!decoded) {
  return res.status(401).json({ error: 'Invalid or expired token' });
}

// 3. Attach user info to the request object for downstream handlers
req.user = decoded; // { userId: '...', username: '...' }
next(); // pass to the next middleware or route handler
}

module.exports = { authMiddleware, verifyToken };

// _____

// Using it in routes:
const { authMiddleware } = require('./authMiddleware');

// Apply to a single route
app.get('/profile', authMiddleware, (req, res) => {
  res.json({ message: `Hello ${req.user.username}!` });
});

// Apply to all routes in a router
router.use(authMiddleware);
router.get('/dashboard', (req, res) => { /* protected */ });
router.post('/post', (req, res) => { /* protected */ });
```

Complete Register and Login Implementation

```
auth.js - Complete Implementation

const express = require('express');
const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const User = require('./models/User');
const router = express.Router();

// POST /auth/register
router.post('/register', async (req, res, next) => {
  try {
    const { username, email, password } = req.body;

    // 1. Validate input
```



```
    if (!username || !email || !password) {
      return res.status(400).json({ error: 'All fields required' });
    }
    if (password.length < 8) {
      return res.status(400).json({ error: 'Password must be 8+ chars' });
    }

    // 2. Check if user already exists
    const existing = await User.findOne({ $or: [{ username }, { email }] });
    if (existing) {
      return res.status(409).json({ error: 'Username or email already taken'
    });
    }

    // 3. Hash the password
    const passwordHash = await bcrypt.hash(password, 12);

    // 4. Save user to DB
    const user = await User.create({ username, email, passwordHash });

    // 5. Generate JWT
    const token = jwt.sign({ userId: user._id, username },
process.env.JWT_SECRET, { expiresIn: '7d' });

    res.status(201).json({ token, user: { id: user._id, username, email } });
  } catch (err) {
    next(err);
  }
});

// POST /auth/login
router.post('/login', async (req, res, next) => {
  try {
    const { username, password } = req.body;

    // 1. Find user (include passwordHash which is normally excluded)
    const user = await User.findOne({ username }).select('+passwordHash');
    if (!user) {
      // Important: same error for 'wrong username' and 'wrong password'
      // This prevents attackers from knowing which is wrong
      return res.status(401).json({ error: 'Invalid credentials' });
    }
  }
```

```
// 2. Compare password
const isMatch = await bcrypt.compare(password, user.passwordHash);
if (!isMatch) {
  return res.status(401).json({ error: 'Invalid credentials' });
}

// 3. Generate token
const token = jwt.sign({ userId: user._id, username: user.username },
  process.env.JWT_SECRET, { expiresIn: '7d' });

res.json({ token, user: { id: user._id, username: user.username } });
} catch (err) {
  next(err);
}
});

module.exports = router;
```

Token Storage on the Client

Where should the client store the JWT? Two main options:

localStorage: Easy to use. Vulnerable to XSS (Cross-Site Scripting) attacks — malicious scripts can read it.

HttpOnly Cookies: The browser stores the cookie automatically. JavaScript cannot read it. Much more secure.

For most beginners and projects: localStorage is fine. For production apps with security requirements: use HttpOnly cookies.

CHAPTER 8**WebSockets — Real-Time Communication***From polling to persistent connections: building live features*

The Problem With HTTP for Real-Time Apps

HTTP follows a strict request/response model — the client always initiates. The server cannot push data to the client unprompted. For a chess game, this is a problem: how does Player B know that Player A just moved a piece?

The Old Solution: Polling

Before WebSockets, developers used polling — repeatedly asking the server 'has anything changed?':

The Polling Anti-Pattern

```
// Short polling — ask every 2 seconds
setInterval(async () => {
  const response = await fetch('/api/game/42/latest-move');
  const data = await response.json();
  if (data.newMove) updateBoard(data.newMove);
}, 2000);

// Problems:
// - 2 second delay at minimum
// - Wastes bandwidth (most polls return 'nothing new')
// - Server gets hammered with empty requests
// - 1000 users = 500 requests/second doing nothing useful
```

The WebSocket Solution

WebSockets establish a persistent, bidirectional connection. The protocol starts as HTTP then upgrades:

WebSocket Handshake (HTTP Upgrade)

```
// Client sends an 'upgrade' request:
GET /ws HTTP/1.1
Host: chess.example.com
Upgrade: websocket
Connection: Upgrade
```

```
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==

// Server responds:
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=

// Connection is now upgraded — both sides can send messages any time
// No more request/response — just an open pipe
```

Feature	HTTP	WebSocket
Direction	Client → Server only (client initiates)	Both directions (full-duplex)
Connection	Opens and closes per request	Stays open until explicitly closed
Latency	Overhead of new connection each time	Very low — connection already open
Server push	Not possible natively	Yes — server sends any time
Best for	CRUD APIs, web pages	Chat, games, live dashboards

The 'ws' Library — WebSockets in Node.js

The 'ws' package is the most popular Node.js WebSocket library. Install it with: `npm install ws`

Basic WebSocket Server

```
Basic WebSocket Server

const WebSocket = require('ws');

// Create a WebSocket server on port 8080
const wss = new WebSocket.Server({ port: 8080 });

wss.on('connection', (ws) => {
  // Fires every time a new client connects
  console.log('New client connected. Total:', wss.clients.size);

  // Send a welcome message to this specific client
  ws.send(JSON.stringify({ type: 'WELCOME', message: 'Connected to server!' }));
});
```

```
// Handle messages FROM this client
ws.on('message', (rawData) => {
  const message = JSON.parse(rawData.toString());
  console.log('Received:', message);

  // Echo it back
  ws.send(JSON.stringify({ type: 'ECHO', data: message }));
});

// Handle disconnection
ws.on('close', (code, reason) => {
  console.log(`Client disconnected. Code: ${code}, Reason: ${reason}`);
});

// Handle errors
ws.on('error', (err) => {
  console.error('WebSocket error:', err.message);
});
});

console.log('WebSocket server running on ws://localhost:8080');
```

WebSocket Client (Browser)

```
Browser WebSocket Client

// In the browser — no npm needed, WebSocket is built-in

const socket = new WebSocket('ws://localhost:8080');

// Connection opened
socket.addEventListener('open', () => {
  console.log('Connected to server!');
  // Send a message
  socket.send(JSON.stringify({ type: 'HELLO', text: 'Hi server!' }));
});

// Message received
socket.addEventListener('message', (event) => {
  const message = JSON.parse(event.data);
  console.log('Server says:', message);
});
```

```
});

// Connection closed
socket.addEventListener('close', () => {
  console.log('Disconnected from server');
});

// Error
socket.addEventListener('error', (err) => {
  console.error('WebSocket error:', err);
});

// Sending messages
socket.send(JSON.stringify({ type: 'MESSAGE', text: 'Hello!' }));

// Closing connection
socket.close();
```

Broadcasting — Sending to All Clients

The most common WebSocket pattern is broadcasting: when one client sends a message, relay it to all connected clients. This is the core of any chat or multiplayer game:

Broadcasting to All Clients

```
wss.on('connection', (ws) => {

  ws.on('message', (rawData) => {
    const message = JSON.parse(rawData.toString());

    if (message.type === 'CHAT_MESSAGE') {
      // Broadcast to ALL connected clients
      const broadcast = JSON.stringify({
        type: 'CHAT_MESSAGE',
        from: message.username,
        text: message.text,
        timestamp: new Date().toISOString(),
      });

      wss.clients.forEach((client) => {
        // Only send to OPEN connections
        if (client.readyState === WebSocket.OPEN) {
          client.send(broadcast);
        }
      });
    }
  });
});
```

```
    }
    });
  }
});
});

// wss.clients is a Set of all currently connected WebSocket objects
// WebSocket.OPEN means the connection is alive and ready
```

Rooms — Sending to a Subset of Clients

In a chess game, you do not want to broadcast every move to all users — only to the two players in that specific game. You implement 'rooms' manually by tracking which client belongs to which game:

WebSocket Rooms

```
// Map<gameId, Set<WebSocket>> - which clients are in which game
const gameRooms = new Map();

function joinRoom(gameId, ws) {
  if (!gameRooms.has(gameId)) {
    gameRooms.set(gameId, new Set());
  }
  gameRooms.get(gameId).add(ws);
  ws.gameId = gameId; // remember which room this socket is in
}

function leaveRoom(ws) {
  if (ws.gameId && gameRooms.has(ws.gameId)) {
    gameRooms.get(ws.gameId).delete(ws);
  }
}

function broadcastToRoom(gameId, message) {
  const room = gameRooms.get(gameId);
  if (!room) return;
  const json = JSON.stringify(message);
  room.forEach(client => {
    if (client.readyState === WebSocket.OPEN) {
      client.send(json);
    }
  });
}
```

```
}

// — Using rooms in the connection handler —————
wss.on('connection', (ws) => {
  ws.on('message', (rawData) => {
    const msg = JSON.parse(rawData.toString());

    if (msg.type === 'JOIN_GAME') {
      joinRoom(msg.gameId, ws);
      broadcastToRoom(msg.gameId, { type: 'PLAYER_JOINED', userId: ws.userId
    });
    }

    if (msg.type === 'MOVE') {
      broadcastToRoom(ws.gameId, {
        type: 'MOVE',
        from: msg.from,
        to: msg.to,
        by: ws.userId
      });
    }
  });

  ws.on('close', () => leaveRoom(ws));
});
```

WebSocket Connection States

Before sending, always check the connection state to avoid errors:

```
Connection States

// WebSocket has 4 readyState values:
// WebSocket.CONNECTING = 0 — handshake in progress
// WebSocket.OPEN       = 1 — connected, can send/receive
// WebSocket.CLOSING    = 2 — close handshake in progress
// WebSocket.CLOSED     = 3 — connection closed

function safeSend(ws, message) {
  if (ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify(message));
  } else {
    console.warn('Tried to send to a non-open socket:', ws.readyState);
  }
}
```



```
}  
}
```

Heartbeats — Keeping Connections Alive

Some networks (NAT, firewalls, mobile networks) automatically close idle connections after a timeout. Heartbeats are periodic pings to keep the connection alive and detect dead clients:

Heartbeat Implementation

```
const HEARTBEAT_INTERVAL = 30000; // 30 seconds  
  
wss.on('connection', (ws) => {  
  ws.isAlive = true;  
  
  // Client responds to pings with pongs automatically (ws library handles this)  
  ws.on('pong', () => {  
    ws.isAlive = true; // client is still alive  
  });  
});  
  
// Every 30 seconds, ping all clients  
const heartbeat = setInterval(() => {  
  wss.clients.forEach((ws) => {  
    if (!ws.isAlive) {  
      // Did not respond to last ping — assume dead, terminate  
      console.log('Terminating dead connection');  
      return ws.terminate();  
    }  
    ws.isAlive = false; // reset — if pong comes back, set to true  
    ws.ping();          // send ping  
  });  
, HEARTBEAT_INTERVAL);  
  
// Clean up interval when server closes  
wss.on('close', () => clearInterval(heartbeat));
```

Integrating WebSocket Server With Express

In production, you usually want both HTTP (for your REST API) and WebSocket on the same port. You can attach the WebSocket server to an existing HTTP server:

Express + WebSocket on Same Port

```
const express = require('express');
const http = require('http');
const WebSocket = require('ws');

const app = express();
app.use(express.json());

// Define REST routes as normal
app.get('/health', (req, res) => res.json({ status: 'ok' }));

// Create HTTP server from Express app
const server = http.createServer(app);

// Attach WebSocket server to the SAME HTTP server
const wss = new WebSocket.Server({ server });

wss.on('connection', (ws) => {
  ws.send(JSON.stringify({ type: 'CONNECTED' }));
});

// Start ONE server that handles both HTTP and WebSocket
server.listen(3000, () => {
  console.log('HTTP + WebSocket server on port 3000');
  // REST: http://localhost:3000/health
  // WS:   ws://localhost:3000
});
```

CHAPTER 9**WebSockets + JWT Authentication***Securing real-time connections the right way*

Why Authenticate WebSocket Connections?

Without authentication, anyone who knows your WebSocket URL can connect. In a chess game, that means anyone could inject fake moves, read private game state, or flood your server with connections. You must authenticate WebSocket connections just like HTTP routes.

Strategy: Token in the Query Parameter

WebSocket connections start as HTTP requests, so they can carry data in the URL query string. The client passes the JWT as a query parameter when opening the connection:

Client: Sending JWT via Query Param

```
// — CLIENT (Browser) —————

const token = localStorage.getItem('authToken'); // JWT from login

// Pass token in the URL
const socket = new WebSocket(`ws://localhost:3000?token=${token}`);

socket.addEventListener('open', () => {
  console.log('Authenticated WebSocket connected!');
});

socket.addEventListener('message', (event) => {
  const msg = JSON.parse(event.data);
  if (msg.type === 'AUTH_FAILED') {
    console.error('Authentication failed - token invalid or expired');
    socket.close();
  }
});
```

Server: Verifying the Token on Connection

Server: JWT Verification on WebSocket Connection

```
const WebSocket = require('ws');
const url = require('url'); // Node.js built-in URL parser
const jwt = require('jsonwebtoken');

const wss = new WebSocket.Server({ server });

wss.on('connection', (ws, req) => {
  // req is the raw HTTP request that initiated the upgrade
  // Parse the query string from the request URL
  const params = new URLSearchParams(req.url.split('?')[1]);
  const token = params.get('token');

  // Verify the token
  if (!token) {
```

```
ws.send(JSON.stringify({ type: 'AUTH_FAILED', error: 'No token' }));
ws.close(1008, 'Authentication required'); // 1008 = Policy Violation
return;
}

let decoded;
try {
  decoded = jwt.verify(token, process.env.JWT_SECRET);
} catch (err) {
  ws.send(JSON.stringify({ type: 'AUTH_FAILED', error: 'Invalid token' }));
  ws.close(1008, 'Invalid token');
  return;
}

// Token is valid! Attach user identity to the WebSocket object
ws.userId = decoded.userId;
ws.username = decoded.username;

console.log(`User ${ws.username} connected via WebSocket`);
ws.send(JSON.stringify({ type: 'AUTH_SUCCESS', userId: ws.userId }));

// Now handle messages as normal, with access to ws.userId
ws.on('message', (rawData) => {
  const msg = JSON.parse(rawData.toString());
  console.log(`Message from ${ws.username}:`, msg);
  // ... handle message
});

ws.on('close', () => {
  console.log(`User ${ws.username} disconnected`);
});
});
```

Complete Authenticated Chess Game WebSocket Handler

Complete Authenticated Chess WebSocket Handler

```
// ws.ts - simplified chess WebSocket handler
```

```
const games = new Map(); // gameId → { board, white, black, moves[] }
const gameRooms = new Map(); // gameId → Set<WebSocket>
```

```
wss.on('connection', (ws, req) => {

  // 1. Authenticate
  const token = new URLSearchParams(req.url.split('?')[1]).get('token');
  try {
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    ws.userId = decoded.userId;
  } catch {
    ws.close(1008, 'Unauthorized');
    return;
  }

  ws.on('message', async (rawData) => {
    const msg = JSON.parse(rawData.toString());

    // 2. Handle JOIN
    if (msg.type === 'JOIN') {
      const { gameId } = msg;

      // Add client to the room
      if (!gameRooms.has(gameId)) gameRooms.set(gameId, new Set());
      gameRooms.get(gameId).add(ws);
      ws.gameId = gameId;

      // Load game history from DB and replay all moves
      const moves = await Move.find({ gameId }).sort({ createdAt: 1 });
      ws.send(JSON.stringify({ type: 'GAME_STATE', moves }));
    }

    // 3. Handle MOVE
    if (msg.type === 'MOVE') {
      const { gameId, from, to } = msg;

      // Validate it is this player's turn
      const game = games.get(gameId);
      const isWhiteTurn = game.board.turn() === 'w';
      const isWhite = game.white.toString() === ws.userId.toString();
      if (isWhiteTurn !== isWhite) {
        ws.send(JSON.stringify({ type: 'ERROR', error: 'Not your turn' }));
        return;
      }
    }
  });
});
```

```
// Validate the chess move
const result = game.board.move({ from, to });
if (!result) {
  ws.send(JSON.stringify({ type: 'ERROR', error: 'Illegal move' }));
  return;
}

// Save move to DB
await Move.create({ gameId, from, to, by: ws.userId });

// Broadcast updated state to all players in this room
broadcastToRoom(gameId, {
  type: 'MOVE_MADE',
  from, to,
  fen: game.board.fen(), // full board position
  isGameOver: game.board.isGameOver(),
});
}
});

ws.on('close', () => {
  if (ws.gameId) gameRooms.get(ws.gameId)?.delete(ws);
});
});
```

Security Consideration: Token Exposure in URLs

Passing tokens in query parameters means the token appears in server access logs, browser history, and referrer headers. This is a minor security concern.

A safer alternative: send the token as the first WebSocket message after connecting, and close the connection if authentication fails. However, this adds complexity since you cannot reject the connection before it is established.

For most applications, query-parameter tokens are acceptable. For high-security applications (banking, medical), consider custom WebSocket-level auth protocols.

CHAPTER 10

RabbitMQ — Message Queues*Decoupling services with asynchronous messaging*

Why Message Queues?

Imagine two services: Service A (web server) and Service B (game engine). Service A needs Service B to process a chess move. The naive approach is a direct HTTP call — A calls B and waits. This works, but has problems:

- **Tight coupling:** A must know B's address. If B moves or changes port, A breaks.
- **No resilience:** If B is down when A calls it, the move is lost.
- **No load levelling:** If 1000 moves arrive in one second, all 1000 are slammed at B simultaneously. B might crash.

A message queue sits between A and B. A sends the move to the queue and continues. B reads from the queue when it is ready. The queue absorbs spikes, survives outages, and decouples the services completely.

Core RabbitMQ Concepts

Concept	What It Is	Real-World Analogy
Producer	Code that sends messages	Person writing a letter
Consumer	Code that receives and processes messages	Person reading the mailbox
Queue	Buffer that stores messages until consumed	The mailbox itself
Exchange	Router that decides which queue(s) to send to	Post office sorting room
Binding	Rule connecting an exchange to a queue	Delivery route from sorting room to mailbox
Message	Data being sent (usually JSON string)	The letter itself
Channel	Lightweight connection inside a connection	Lane on a highway

Exchange Types — How Messages Are Routed

Type	Routing Logic	Use Case
direct	Message goes to queue with matching routing key	Chess moves → specific game engine
fanout	Message goes to ALL bound queues (broadcast)	Game result → leaderboard + stats + notifications
topic	Wildcard routing key matching (logs.*, *.error)	Log routing: logs.chess.error → error queue only
headers	Route based on message header values	Complex routing conditions

Setting Up RabbitMQ with Docker

Running RabbitMQ

```
# Pull and run RabbitMQ with the management plugin
docker run -d \
  --name rabbitmq \
  -p 5672:5672 \   # AMQP port (applications connect here)
  -p 15672:15672 \  # Management UI (browser interface)
  rabbitmq:3-management

# Management UI: http://localhost:15672
# Default login: guest / guest

# Check it is running
docker ps

# Install the amqplib package in your Node.js project
npm install amqplib
```

Basic Producer — Sending Messages

Producer: Sending Messages to a Queue

```
const amqp = require('amqplib');

async function createProducer() {
  // 1. Connect to RabbitMQ
  const connection = await amqp.connect('amqp://localhost');

  // 2. Create a channel (lightweight multiplexing over the connection)
```



```
const channel = await connection.createChannel();

// 3. Declare the queue (creates it if it does not exist)
const QUEUE_NAME = 'chess-moves';
await channel.assertQueue(QUEUE_NAME, {
  durable: true, // survives RabbitMQ restarts
});

// 4. Send a message
const move = { gameId: 'game-123', from: 'e2', to: 'e4', playerId: 'alice' };
const messageBuffer = Buffer.from(JSON.stringify(move));

channel.sendToQueue(QUEUE_NAME, messageBuffer, {
  persistent: true, // message survives RabbitMQ restarts
});

console.log('Move sent:', move);

// 5. Close connection (in real apps, reuse connection and channel)
setTimeout(() => { connection.close(); }, 500);
}

createProducer().catch(console.error);
```

Basic Consumer — Receiving Messages

Consumer: Receiving and Acknowledging Messages

```
const amqp = require('amqplib');

async function createConsumer() {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();

  const QUEUE_NAME = 'chess-moves';
  await channel.assertQueue(QUEUE_NAME, { durable: true });

  // Only take 1 message at a time (don't overwhelm this consumer)
  channel.prefetch(1);

  console.log('Waiting for chess moves...');
```

```
channel.consume(QueueName, async (msg) => {
  if (!msg) return; // null means consumer was cancelled

  const move = JSON.parse(msg.content.toString());
  console.log('Processing move:', move);

  try {
    // Process the move (validate chess rules, update game state, save to DB)
    await processChessMove(move);

    // ACK — tell RabbitMQ 'I processed this successfully, remove it'
    channel.ack(msg);
  } catch (err) {
    console.error('Failed to process move:', err);

    // NACK — tell RabbitMQ 'I failed, put this back in the queue'
    // requeue: true → back in queue for another attempt
    // requeue: false → discard (or send to dead letter queue)
    channel.nack(msg, false, true);
  }
}, {
  noAck: false, // CRITICAL: manual acknowledgement mode
  // noAck: true would remove messages immediately — DANGEROUS!
});

createConsumer().catch(console.error);
```

The noAck Trap — Read This Carefully

`noAck: true` means 'automatically acknowledge every message as soon as it is delivered'. The message is removed from the queue the moment the consumer receives it.

If your consumer crashes while processing the message (e.g., database connection drops, bug throws an exception), the message is **PERMANENTLY LOST**. For a chess game, this means a player's move disappears — the game state becomes corrupt.

Always use `noAck: false` (the default). Send `channel.ack(msg)` only **AFTER** successful processing. Send `channel.nack(msg, false, true)` on failure to re-queue. This guarantees at-least-once delivery.

Work Queues — Load Balancing Across Multiple Consumers

When one consumer cannot keep up with the message volume, you run multiple consumers. RabbitMQ distributes messages among them (round-robin by default, or according to prefetch):

Multiple Consumers (Work Queue Pattern)

```
// Run this same consumer script multiple times in different terminals:
// Terminal 1: node consumer.js
// Terminal 2: node consumer.js
// Terminal 3: node consumer.js

// RabbitMQ will distribute messages among the 3 consumers.
// With prefetch(1), each consumer gets one message at a time.
// Busy consumers won't get new messages until they ack the current one.

channel.prefetch(1); // fair dispatch — don't give more until acked

// — Producer side: same queue, nothing changes —————
channel.sendToQueue('chess-moves', Buffer.from(JSON.stringify(move)));

// RabbitMQ handles distribution automatically.
// If consumer 1 is processing a complex move, message 2 goes to consumer 2.
```

Fanout Exchange — Broadcasting to All Consumers

Fanout Exchange Pattern

```
// — PRODUCER: Publish to a fanout exchange —————

async function setupFanoutProducer() {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();

  // Declare a fanout exchange
  await channel.assertExchange('game-events', 'fanout', { durable: true });

  const gameResult = {
    gameId: 'game-123',
    winner: 'alice',
    loser: 'bob',
    endReason: 'checkmate',
    duration: 1842,
  };
};
```

```
// Publish to exchange (not directly to queue)
// Routing key is ignored for fanout, but required by the API
channel.publish('game-events', '', Buffer.from(JSON.stringify(gameResult)));
console.log('Game result published');
}

// — CONSUMER: Subscribe with an exclusive queue —————

async function subscribeEvents(serviceName) {
  const connection = await amqp.connect('amqp://localhost');
  const channel = await connection.createChannel();

  // Declare the same fanout exchange
  await channel.assertExchange('game-events', 'fanout', { durable: true });

  // Create an EXCLUSIVE, server-named queue
  // exclusive: true → only this connection can use it, auto-deleted when
  // closed
  // '' name → RabbitMQ assigns a unique name like 'amq.gen-XYZ'
  const { queue: queueName } = await channel.assertQueue('', { exclusive: true });

  // Bind the queue to the fanout exchange
  // Every message sent to the exchange will be delivered to this queue
  await channel.bindQueue(queueName, 'game-events', '');

  console.log(`${serviceName} listening on queue: ${queueName}`);

  channel.consume(queueName, (msg) => {
    const event = JSON.parse(msg.content.toString());
    console.log(`[${serviceName}] Received:`, event);
    // Each service does its own thing:
    // leaderboard-service → update rankings
    // stats-service → update player statistics
    // notification-service → send email to players
    channel.ack(msg);
  }, { noAck: false });
}

// Run three separate services — all get every game result:
subscribeEvents('leaderboard-service');
subscribeEvents('stats-service');
```

```
subscribeEvents('notification-service');
```

💡 Why Exclusive Queues for subscribeEvents?

Each WebSocket server instance needs its OWN copy of game events — not shared. If three WebSocket servers shared one queue, each event would only reach ONE of them (work queue behavior). Players connected to the other two would never see the result.

Exclusive queues give each subscriber their own private queue. All bind to the same fanout exchange and all get every message.

When a WebSocket server shuts down, its exclusive queue is automatically deleted. No manual cleanup, no queue buildup.

This combination (fanout + exclusive queue per consumer) is the pub/sub (publish-subscribe) pattern.

Dead Letter Queues — Handling Failed Messages

What happens when a message fails processing repeatedly? It should not loop forever. A Dead Letter Queue (DLQ) is a separate queue that receives messages that have been rejected or expired:

Dead Letter Queue Setup

```
// Set up a dead letter queue for failed moves
await channel.assertQueue('chess-moves-failed', { durable: true });

// Configure the main queue to send failures to the DLQ
await channel.assertQueue('chess-moves', {
  durable: true,
  arguments: {
    'x-dead-letter-exchange': '', // default exchange
    'x-dead-letter-routing-key': 'chess-moves-failed', // target queue
    'x-message-ttl': 60000, // messages expire after 60 seconds if not
    consumed
  }
});

// Consumer: on failure, nack without requeue → goes to DLQ
channel.consume('chess-moves', async (msg) => {
  try {
    await processMove(JSON.parse(msg.content.toString()));
    channel.ack(msg);
  } catch (err) {
    // Don't requeue — send to dead letter queue instead
  }
});
```

```
    channel.nack(msg, false, false); // last false = don't requeue
  }
});

// Separately, monitor the DLQ to alert engineers or retry later
channel.consume('chess-moves-failed', (msg) => {
  const move = JSON.parse(msg.content.toString());
  console.error('FAILED MOVE (alert engineering!):', move);
  // Could: send to Slack, email, Sentry, etc.
  channel.ack(msg);
});
```

Connection Resilience — Reconnecting on Failure

Connection Resilience

```
// RabbitMQ connections can drop. Always implement reconnection logic.

async function connectWithRetry(url, retries = 5, delay = 3000) {
  for (let attempt = 1; attempt <= retries; attempt++) {
    try {
      const connection = await amqp.connect(url);
      console.log('Connected to RabbitMQ');

      // Handle unexpected disconnections
      connection.on('error', (err) => {
        console.error('RabbitMQ connection error:', err.message);
      });
      connection.on('close', () => {
        console.log('RabbitMQ connection closed. Reconnecting...');
        setTimeout(() => connectWithRetry(url), delay);
      });

      return connection;
    } catch (err) {
      console.error(`Attempt ${attempt}/${retries} failed:`, err.message);
      if (attempt < retries) {
        await new Promise(res => setTimeout(res, delay));
      }
    }
  }
  throw new Error('Could not connect to RabbitMQ after retries');
}
```

CHAPTER 11

Microservices Architecture

Splitting a monolith into independent, scalable services

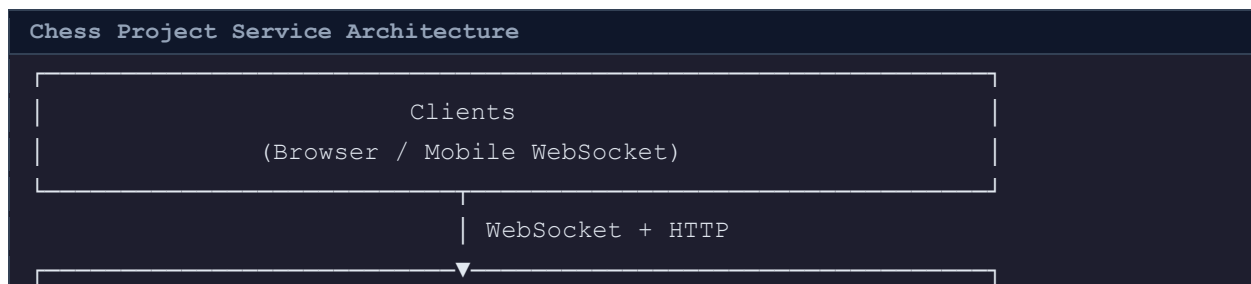
Monolith vs Microservices

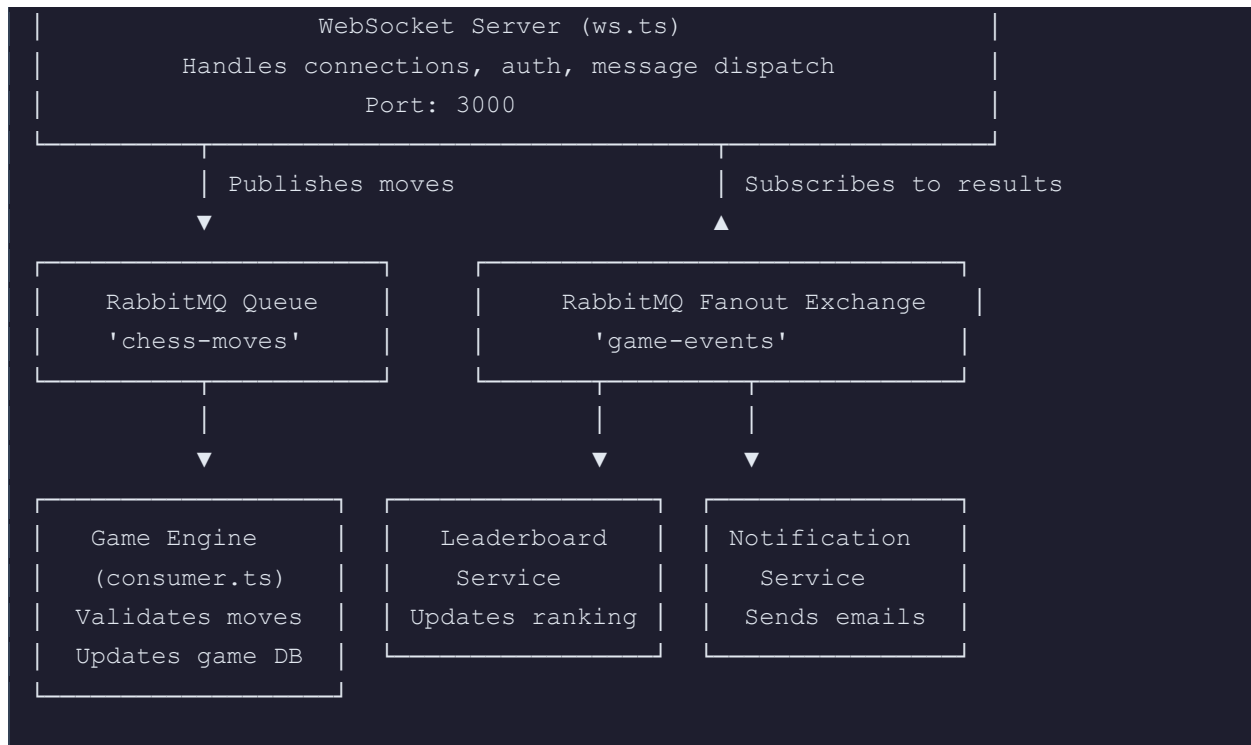
A monolith is a single application that does everything. One codebase, one deployment, one database. Simple to start with — but as the application grows, it becomes harder to maintain, test, and scale.

Aspect	Monolith	Microservices
Deployment	Deploy entire app for any change	Deploy only the changed service
Scaling	Scale the whole app even for one feature	Scale only the bottleneck service
Failure	One bug can crash everything	Failures are isolated per service
Tech stack	Must use one language/framework	Each service can use best tool
Complexity	Simple to develop initially	Complex inter-service communication
Team size	Good for small teams	Good for large teams (own a service)
Good starting?	YES — start monolith, extract later	Can over-engineer early apps

The Chess Project's Microservices Architecture

The chess project splits into these services:





The Game Engine — consumer.ts Deep Dive

The game engine is a pure consumer — it reads chess moves from RabbitMQ, validates them, and publishes results. Here is the complete flow:

```

Game Engine Consumer (consumer.ts)

// consumer.ts - simplified game engine consumer

import amqp from 'amqplib';
import { Chess } from 'chess.js';
import mongoose from 'mongoose';
import { Move } from '../models/Move';
import { Game } from '../models/Game';

// In-memory cache: gameId → Chess instance (current board state)
const gameBoards = new Map<string, Chess>();

async function getOrCreateBoard(gameId: string): Promise<Chess> {
  // Check in-memory cache first (fast)
  if (gameBoards.has(gameId)) return gameBoards.get(gameId)!;

  // Not in memory (first time or after restart) - load from MongoDB
  const chess = new Chess(); // fresh board

```



```
const moves = await Move.find({ gameId }).sort({ createdAt: 1 });

// Replay all historical moves to reconstruct current state
for (const move of moves) {
  chess.move({ from: move.from, to: move.to });
}

gameBoards.set(gameId, chess);
return chess;
}

async function startEngine() {
  await mongoose.connect(process.env.MONGO_URI!);
  const connection = await amqp.connect(process.env.RABBITMQ_URL!);
  const channel = await connection.createChannel();

  // Input queue: receive moves to process
  await channel.assertQueue('chess-moves', { durable: true });
  channel.prefetch(1); // process one move at a time (maintains order!)

  // Output exchange: broadcast results
  await channel.assertExchange('game-events', 'fanout', { durable: true });

  console.log('Game engine ready. Waiting for moves...');

  channel.consume('chess-moves', async (msg) => {
    if (!msg) return;

    const { gameId, from, to, playerId } = JSON.parse(msg.content.toString());

    try {
      const chess = await getOrCreateBoard(gameId);
      const game = await Game.findById(gameId);

      // Validate it is the player's turn
      const isWhiteTurn = chess.turn() === 'w';
      const isPlayerWhite = game!.whitePlayer.toString() === playerId;
      if (isWhiteTurn !== isPlayerWhite) {
        throw new Error('Not this player\'s turn');
      }

      // Validate and make the move
```

```
const moveResult = chess.move({ from, to, promotion: 'q' });
if (!moveResult) throw new Error('Illegal move');

// Persist the move to MongoDB
await Move.create({ gameId, from, to, playerId, fen: chess.fen() });

// Check for game over
let gameOverData = null;
if (chess.isGameOver()) {
  // game.turn() returns whose turn it is — in checkmate, the LOSER
  const loserColor = chess.turn();
  const loser = loserColor === 'w' ? game!.whitePlayer :
game!.blackPlayer;
  const winner = loserColor === 'w' ? game!.blackPlayer :
game!.whitePlayer;

  await Game.findByIdAndUpdate(gameId, { status: 'finished', winner });
  gameOverData = { winner, loser, reason: chess.isCheckmate() ?
'checkmate' : 'draw' };

  gameBoards.delete(gameId); // clean up memory
}

// Publish result to all interested services via fanout exchange
const result = { gameId, from, to, fen: chess.fen(), isGameOver:
!!gameOverData, gameOverData };
channel.publish('game-events', '', Buffer.from(JSON.stringify(result)));

channel.ack(msg); // success — remove from queue
} catch (err: any) {
  console.error('Move processing failed:', err.message);

  // Publish error event so WebSocket server can tell the player
  channel.publish('game-events', '', Buffer.from(JSON.stringify({
    gameId, error: err.message, type: 'MOVE_REJECTED'
  })));

  channel.ack(msg); // ack anyway to prevent infinite retry loop
  // (A nack here would cause the same invalid move to loop forever)
}
}, { noAck: false });
}
```

```
startEngine().catch(console.error);
```

Consistent Hashing — Why Moves Stay on the Same Engine

The game engine maintains board state in memory for performance. If moves from the same game went to different engine instances, each instance would have an incomplete history and could not validate moves correctly.

The solution is to ensure all moves for a given game always go to the same engine instance. You can achieve this with multiple named queues — one per engine instance:

```
Consistent Game Routing

// Producer (WebSocket server): route moves by game ID

const NUM_ENGINE_INSTANCES = 3;

function getQueueForGame(gameId: string): string {
  // Simple hash: sum of char codes modulo number of instances
  let hash = 0;
  for (const char of gameId) {
    hash = (hash + char.charCodeAt(0)) % NUM_ENGINE_INSTANCES;
  }
  return `chess-engine-${hash}`; // 'chess-engine-0', 'chess-engine-1', or 'chess-engine-2'
}

// All moves for game 'game-abc123' always go to the same queue
const queueName = getQueueForGame(move.gameId);
channel.sendToQueue(queueName, Buffer.from(JSON.stringify(move)));

// — Engine instances —————
// Engine 0: listens on 'chess-engine-0'
// Engine 1: listens on 'chess-engine-1'
// Engine 2: listens on 'chess-engine-2'
// Each engine handles its subset of games — always the same games
```

Service Communication Patterns

In microservices, services communicate in two main ways:

Pattern	How It Works	When to Use
---------	--------------	-------------

Synchronous (HTTP/RPC)	Service A waits for Service B's response	When you need an immediate answer (e.g., validate a payment before confirming an order)
Asynchronous (Queue)	Service A sends message, continues; Service B processes later	When response is not needed immediately (e.g., send email after registration, process analytics)

Service Discovery and Environment Variables

Each microservice needs to know where others are running. In development, you hardcode localhost URLs. In production, you use environment variables and service discovery:

Docker Compose for Microservices

```
// .env file for WebSocket server service
PORT=3000
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/chess
JWT_SECRET=your-very-long-random-secret-key-here
RABBITMQ_URL=amqp://rabbitmq:5672 # 'rabbitmq' = Docker service name

// .env file for game engine service
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/chess
RABBITMQ_URL=amqp://rabbitmq:5672

// docker-compose.yml - run all services together
version: '3.8'
services:
  rabbitmq:
    image: rabbitmq:3-management
    ports:
      - '5672:5672'
      - '15672:15672'

  websocket-server:
    build: ./websocket-server
    ports:
      - '3000:3000'
    env_file: ./websocket-server/.env
    depends_on:
      - rabbitmq

  game-engine:
    build: ./game-engine
```

```
env_file: ./game-engine/.env
depends_on:
  - rabbitmq
deploy:
  replicas: 3    # run 3 engine instances
```

The Complete Move Processing Flow

Let us trace a chess move from the moment a player clicks a piece to the moment both players see the updated board:

1. **Player clicks:** Browser sends WebSocket message: { type: 'MOVE', gameId: 'abc', from: 'e2', to: 'e4' }
2. **WebSocket server receives:** Verifies JWT from the connection (ws.userId). Confirms the player is in this game.
3. **Publishes to RabbitMQ:** Sends move to 'chess-engine-N' queue based on game ID hash.
4. **WebSocket server returns immediately:** Does NOT wait for the engine — continues handling other connections.
5. **Game engine consumes:** Reads the message from its queue. Reconstructs board state (memory or MongoDB). Validates the move with chess.js.
6. **Engine saves to MongoDB:** Writes Move document with gameId, from, to, fen, timestamp.
7. **Engine publishes result:** Sends { gameId, fen, isGameOver } to 'game-events' fanout exchange.
8. **All services receive:** WebSocket server gets it via its exclusive queue. Leaderboard service gets its copy. Notification service gets its copy.
9. **WebSocket server broadcasts:** Sends MOVE_MADE event to both players connected to this game via the game room.
10. **Both browsers update:** Chess board re-renders with the new position.

What You Now Understand

Every concept in the study plan is now a tool in your toolkit, not just a word.

HTTP and REST: the language services speak to each other and to clients.

Express: the framework that makes writing HTTP servers simple.

MongoDB and Mongoose: where data lives permanently and how to structure it.

Authentication: how to verify identity with bcrypt + JWT + middleware.

WebSockets: persistent connections for real-time features.

RabbitMQ: message queues that decouple services and guarantee delivery.

Microservices: splitting a complex system into independent, scalable pieces.

The chess project: a real-world example that uses ALL of these together.